Doc. no.:	P3060R0
Date:	2023-11-22
Audience:	LEWG
Reply-to:	Weile Wei <weilewei09@gmail.com>

Add std::ranges::upto(n)

Abstract

This proposal suggests adding `std::ranges::upto(n)` to the C++ Standard Library as a range adaptor that generates a sequence of integers from `0` to `n-1`.

Motivation

Typically, generating such a sequence of integers involves `std::views::iota` combined with `std::views::take`, or manual iteration, which can be cumbersome and not straightforward, particularly with unsigned types like container sizes. Consider the following typical usage:

before	`for (int i : std::views::iota(0) | std::views::take(10)) { std::cout << i << ' '; } std::cout << '\n';`
after	`for (int i : std::views::upto(10)) { std::cout << i << ' '; } std::cout << '\n';`

`std::ranges::upto(n)` eases this pattern by providing a straightforward and type-safe method to generate integer sequences, improving code readability and minimizing repetitive code.

Implementation

```
namespace std::ranges {
    inline constexpr auto upto = [] <std::integral I> (I n) {
        return std::views::iota(I{}, n);
    };
}
```

Test cases verified on various platforms

Test case:

```
int main() {
    constexpr auto up_to_ten = std::ranges::upto(10);

    for (auto i : up_to_ten) {
        std::cout << i << ' ';
    }
    std::cout << std::endl;
    return 0;
}
```

Expected output:

```
0 1 2 3 4 5 6 7 8 9
```

The implementation is confirmed to work with clang version 16.0.0+, gcc version 11.1+, and msvc v19.32+: Compiler Explorer T7YoMzsdG (https://godbolt.org/z/T7YoMzsdG)

Background

Two preceding proposals have provided fundation for `std::ranges::upto(n)`:

1. *P2214R2: A Plan for C++26 Ranges*[1] highlights the issue with `views::iota(0, r.size())` not compiling due to mismatched types. `std::ranges::views::iota` requires both arguments to be of the same type, or at least commonly comparable. This becomes problematic when comparing `int` (often 32-bit) with `std::size_t` (often 64-bit), which is usually wider on 64-bit systems. An example illustrating this issue is available at Compiler Explorer TGzP4be7K (https://godbolt.org/z/TGzP4be7K).

```
std::vector vec(10, 0);
auto res = std::ranges::views::iota(0, vec.size());
```

2. *P1894R0: Proposal of std::upto, std::indices and std::enumerate*[2] proposed an implementation (see below) that our proposal refines. Our approach offers a more intuitive and consistent interface, fitting seamlessly with existing standard library components without adding complexity.

```
// std::upto implementation example
// P1894R0: Proposal of std::upto, std::indices and std::enumerate
namespace std {
    template<typename Int>
    constexpr auto upto(Int&& i) noexcept {
        return std::ranges::iota_view{Int(),std::forward<Int>(i)};
    }
}
```

Technical Decisions

1. Limiting to `std::integral`: This constraint ensures functionality is restricted to integral types, yielding predictable behavior and avoiding the pitfalls of generic types. A more relaxed constraint to `std::default_initializable` and `std::incrementable` would

allow iterators to compile but might introduce undefined behavior. An example illustrating this issue is available at Compiler Explorer hGx1T9sxG (https://godbolt.org/z/hGx1T9sxG):

```
namespace std::ranges {
    // a more relaxed design pattern, compiled but UB
    inline constexpr auto upto2 = []
    <typename T>
    requires std::default_initializable<T> && std::incrementable<T>
        (T n) {
        return std::views::iota(T{}, n);
    };
}

int main() {
    int myint = 10;
    int* ptr = &myint;

    // !!!compiled but undefined behavior!!!
    auto up_to_ten2 = std::ranges::upto2(CustomIterator{ptr});
    for (auto i : up_to_ten2) {
        std::cout << *i << ' ';
    }

    return 0;
}
```

2. Lambda-based Approach: Employing a lambda is consistent with the established range adaptor patterns. Moreoever, the `constexpr` specifier on the lambda ensures that the lambda object is a constant expression.

3. Leveraging Existing `iota_view` Instead of Creating a New `upto_view` : Introducing `upto_view` would mean adding a component similar to what already exists, causing confusion and maintainability issues for users. By leveraging `iota_view` , we simplify the implementation and reuse of what the current C++ Standard Library offers. Additionally, any future optimizations to `iota_view` will automatically benefit `std::ranges::upto` . Therefore, by extending `iota_view` , `std::ranges::upto` becomes more maintainable, efficient, and simple.

Wording

This wording is relative to N4964 (https://open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4964.pdf).

Header <ranges> synopsis [ranges.syn] (http://www.eelis.net/c++draft/ranges.syn)

Insert a new item in the `std::ranges` namespace:

```
namespace std::ranges {
    inline constexpr auto upto = [] <std::integral I> (I n) {
        return std::views::iota(I{}, n);
    };
}
```

New subsection under [range.iota.view] (http://www.eelis.net/c++draft/range.iota.view)

Add a new subsection to describe `std::ranges::upto` :

26.6.4.2.1 upto function [range.upto]

-1- The name `upto` denotes a range adaptor object ([range.adaptor.object] (http://www.eelis.net/c++draft/range.adaptor.object)). The expression `std::ranges::upto(n)` is expression-equivalent to `std::views::iota(I{}, n)` where `I` is the type of `n` .

-2- Constraints: The type `I` shall satisfy the `std::integral` concept.

-3- Returns: A view of the integers in the range [0, n).

-4- Complexity: Constant time.

-5- Remarks: This function template shall not participate in overload resolution unless `std::integral<I>` is true.

References

1. P2214R2 *A Plan for C++26 Ranges.* https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r0.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r0.html) ↩

2. P1894R0 *Proposal of std::upto, std::indices and std::enumerate.* https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1894r0.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1894r0.pdf) ↩